

Presented By Dream Byte Solutions



YOGA LMS PLATFORM

Presented By Dream Byte Solutions

learn.indianyogaassociation.com
(Subdomain of indianyogaassociation.com)

Complete Online Learning Management System

React + Node.js + MongoDB + AWS S3 +
FFmpeg + Razorpay / Stripe

Full Tech Stack | Phase-by-Phase Timeline | User
& Admin Dashboards | Payment Gateway | AWS
Media Storage

February 2026

Presented By Dream Byte Solutions

1. Project Overview

This document is a complete step-by-step guide to build the LMS (Learning Management System) platform for Indian Yoga Association, hosted on the subdomain learn.indianyogaassociation.com. The platform will serve as an online education hub where students can purchase, watch recorded video lectures, join live/scheduled meetings, download course books, and track their learning progress.

The LMS is a fully separate application from the main website (indianyogaassociation.com) and will live on its own subdomain. It shares the same brand identity and will link back and forth with the main site.

What Will the LMS Platform Include?

- Recorded video lectures hosted securely on AWS S3 with CloudFront CDN delivery
- Live / scheduled online meetings (Zoom or Google Meet integration or in-built WebRTC)
- Downloadable course books and PDF study materials stored on AWS S3
- Course purchase via Razorpay (India) and Stripe (International)
- Student / User Dashboard - view enrolled courses, watch lectures, download books
- Admin Dashboard - full control to add/edit/delete courses, videos, books, users, payments
- JWT-based secure authentication (register, login, forgot password)
- Progress tracking - mark lessons as complete, percentage completion per course
- Course categories, search, and filter
- Email notifications - enrollment confirmation, new content alerts

2. Complete Tech Stack

Every technology layer of this project is listed below. The stack is chosen for performance, scalability, and ease of maintenance:

Layer / Category	Technology	Purpose / Usage
Frontend Framework	React.js 18 (Vite)	Build all UI — Student Dashboard, Admin Dashboard, Course Pages
Styling	Bootstrap 5 + Custom CSS	Responsive layout, grid system, pre-built UI components
Animations / UX	Framer Motion	Page transitions, hover effects, loading animations

Presented By Dream Byte Solutions

Routing	React Router v6	Client-side navigation — /login, /dashboard, /course/:id, /admin
State Management	Redux Toolkit / Context API	Global auth state, cart, enrolled courses, notifications
Forms & Validation	React Hook Form + Zod	Login, signup, course upload forms with schema validation
HTTP Requests	Axios	API calls from frontend to Node.js backend
Video Player	Video.js / Plyr	Custom secure video player for recorded lectures
Icons	React Icons + Lucide React	Icon library for UI elements throughout the app
Date & Time	Day.js	Format lecture dates, meeting schedules, enrollment dates
Backend Runtime	Node.js v20 LTS	Server-side JavaScript runtime environment
Backend Framework	Express.js v4	REST API routes — auth, courses, lectures, payments, admin
Database	MongoDB Atlas	Cloud NoSQL database — users, courses, lectures, enrollments
Database ODM	Mongoose	MongoDB schema definition and data validation
Authentication	JWT + bcrypt	Secure login tokens, password hashing
File / Video Upload	Multer + AWS S3 SDK	Upload videos and books directly to AWS S3 bucket
Video Storage	AWS S3 (S3 Bucket)	Primary secure storage for all recorded video lecture files
Video Delivery CDN	AWS CloudFront	Global CDN delivery for fast, low-latency video streaming
Video Processing	FFmpeg (on AWS Lambda / EC2)	Compress, transcode, generate thumbnails from uploaded videos
Book / PDF Storage	AWS S3 (separate prefix/folder)	Secure storage for downloadable course books and materials
Signed URLs	AWS SDK (getSignedUrl)	Time-limited secure URLs so only enrolled students can access content
Email Service	Nodemailer + SendGrid	Enrollment confirmation, password reset, content alerts
Indian Payment	Razorpay	INR payments — UPI, Net Banking, Card for Indian students

Presented By Dream Byte Solutions

International Payment	Stripe	USD / global card payments for international students
Live Meetings	Zoom API / Google Meet API	Schedule and embed live yoga sessions inside the LMS
Config Management	dotenv	Manage all secret environment variables securely
API Security	Helmet + CORS + Rate Limiter	HTTP security headers, CORS policy, brute force protection
Role-Based Access	Custom Middleware (Express)	Student vs Admin role checks on every protected API route
Frontend Deployment	AWS S3 Static Hosting + CloudFront	Host the built React app on AWS with global CDN
Backend Deployment	AWS EC2 / Elastic Beanstalk	Host the Node.js API server on AWS infrastructure
Database Hosting	MongoDB Atlas	Managed cloud database — automated backups and scaling
Process Manager	PM2	Keep Node.js server running — auto-restart on crash
Reverse Proxy	Nginx	Serve frontend, proxy API, handle SSL termination
SSL Certificate	AWS Certificate Manager / Let's Encrypt	HTTPS for learn.indianyogaassociation.com subdomain
Version Control	Git + GitHub	Code versioning and team collaboration
Package Manager	npm / pnpm	Install and manage all project dependencies
Code Quality	ESLint + Prettier	Enforce consistent code style and formatting
API Testing	Postman	Test and debug all backend REST API endpoints

3. Complete Project Timeline

Total Estimated Duration: 6 - 8 weeks for a single developer working full-time. With a team of 2 -3 developers this can be completed in 3 - 4 weeks.

Phase	Phase Name	Timeline (Solo)	Estimated Days
-------	------------	-----------------	----------------

Presented By Dream Byte Solutions

Phase 1	Planning, Setup & Architecture	Week 1	1 – 3 days
Phase 2	Authentication & User System	Week 1 – 2	3 – 7 days
Phase 3	AWS S3 + Video Upload + FFmpeg Pipeline	Week 2 – 3	7 – 14 days
Phase 4	Course & Lecture Management (Backend)	Week 3	14 – 18 days
Phase 5	Student Dashboard (Frontend)	Week 3 – 4	18 – 25 days
Phase 6	Admin Dashboard (Frontend)	Week 4 – 5	25 – 32 days
Phase 7	Payment Integration (Razorpay + Stripe)	Week 5	32 – 36 days
Phase 8	Live Meeting Integration	Week 5 – 6	36 – 40 days
Phase 9	Testing, Bug Fixes & Security	Week 6 – 7	40 – 46 days
Phase 10	Subdomain Deployment & Go Live (AWS)	Week 7 – 8	46 – 52 days

4. Phase-by-Phase Detailed Steps

PHASE 1 — Planning, Setup & Architecture — Week 1 | Days 1–3

- Register and configure the subdomain learn.indianyogaassociation.com in DNS settings — point to AWS CloudFront or EC2
- Create 2 GitHub repositories: `lms-client` (React) and `lms-server` (Node.js)
- Create an AWS account and set up: S3 bucket for videos, S3 bucket for books/PDFs, CloudFront distribution pointing to S3, IAM user with limited S3 access permissions
- Create MongoDB Atlas cluster - create `lms_db` database with collections: `users`, `courses`, `lectures`, `enrollments`, `payments`, `meetings`
- Set up local development environment - Node.js v20, VS Code, Postman
- Draft all API route contracts in Postman before writing any code
- Finalise all environment variables needed for both client and server `.env` files

PHASE 2 — Authentication & User System — Week 1–2 | Days 3–7

- Build User schema in Mongoose: `name`, `email`, `password` (hashed), `role` (student/admin), `enrolledCourses[]`, `profilePhoto`, `createdAt`
- Create POST `/api/auth/register` - validate input, hash password with `bcrypt`, save user, return JWT

Presented By Dream Byte Solutions

- Create POST `/api/auth/login` - verify credentials, return JWT access token (expires 7d) and refresh token (30d)
- Create POST `/api/auth/forgot-password` and POST `/api/auth/reset-password` with secure time-limited reset links sent via SendGrid
- Build `authMiddleware.js` - verify JWT on all protected routes, attach user to `req.user`
- Build `roleMiddleware.js` - block non-admin users from admin routes
- Build React login, register, and forgot password pages using Bootstrap 5 + React Hook Form
- Store JWT token in `httpOnly` cookie (more secure) or `localStorage` - implement auto-refresh on token expiry

PHASE 3 — AWS S3 + Video Upload + FFmpeg Pipeline — Week 2–3 | Days 7–14

- Configure AWS S3 bucket for videos - enable versioning, disable public access, configure CORS for uploads
- Create separate AWS S3 prefix/folder for books and PDFs — configure bucket policies
- Install and configure AWS SDK v3 (`@aws-sdk/client-s3`, `@aws-sdk/s3-request-presigner`) in Node.js server
- Build POST `/api/admin/upload-video` — Multer receives the video file, streams it directly to S3 using multipart upload
- After upload, trigger FFmpeg (on EC2 or Lambda) to: transcode video to HLS format (`.m3u8` + `.ts` chunks), generate thumbnail image, save processed files back to S3
- Build GET `/api/lectures/:id/stream` — generate a time-limited AWS CloudFront signed URL (valid 2 hours) so only enrolled students can stream the video
- Build POST `/api/admin/upload-book` — Multer receives PDF, streams to S3 books folder
- Build GET `/api/lectures/:id/book` — generate signed S3 URL for PDF download (enrolled students only)
- Implement video player on frontend using Video.js with HLS.js — load the signed CloudFront URL

PHASE 4 — Course & Lecture Management (Backend API) — Week 3 | Days 14–18

- Build Course schema: title, description, price (INR), priceUSD, thumbnail, category, level, lectures[], books[], instructor, isPublished, createdAt
- Build Lecture schema: courseId, title, description, videoUrl (S3 HLS URL), thumbnailUrl, duration, order, isPreview (free preview toggle)
- Build Book/Material schema: courseId, title, fileUrl (S3 URL), fileType, createdAt
- Create CRUD API for courses: GET `/api/courses` (all published), GET `/api/courses/:id`, POST `/api/admin/courses`, PUT `/api/admin/courses/:id`, DELETE `/api/admin/courses/:id`
- Create CRUD API for lectures: GET `/api/courses/:id/lectures`, POST `/api/admin/lectures`, PUT `/api/admin/lectures/:id`, DELETE `/api/admin/lectures/:id`

Presented By Dream Byte Solutions

- Create API for books: GET /api/courses/:id/books, POST /api/admin/books, DELETE /api/admin/books/:id
- Build Enrollment schema: userId, courseId, paymentId, enrolledAt, progress{ lectureId, completed }[]
- Create GET /api/user/enrollments - return all courses a logged-in student is enrolled in
- Create POST /api/user/progress - update lecture completion status for a student

PHASE 5 — Student / User Dashboard (Frontend) — Week 3–4 | Days 18–25

- Build main layout for learn.indianyogaassociation.com — Navbar (logo, links to main site, login), Footer, responsive sidebar
- Build Home / Course Listing Page - show all published courses with filter by category and search
- Build Course Detail Page - description, lecture list (with free preview), instructor, price, Enroll button
- Build Student Dashboard (/dashboard) - My Enrolled Courses, progress bars per course, recent activity
- Build Course Player Page (/course/:id/learn) — video player, lecture sidebar, mark-as-complete button, next/previous lecture navigation
- Build Materials / Books Tab inside course player — list all downloadable books with download button (hits signed URL endpoint)
- Build Meetings Tab inside dashboard — list of upcoming and past live meetings for enrolled courses
- Build Profile Page - update name, email, profile photo (upload to S3), change password
- Implement course progress tracking UI - show completion percentage, completed vs total lectures
- Protect all /dashboard routes - redirect to /login if not authenticated

PHASE 6 — Admin Dashboard (Frontend) — Week 4–5 | Days 25–32

- Build Admin Dashboard layout - sidebar with sections: Overview, Courses, Lectures, Books, Students, Payments, Meetings, Settings
- Overview Page - total students enrolled, total revenue, active courses, total lectures, recent enrollments table
- Courses Management - table of all courses with Edit, Publish/Unpublish, Delete actions; Add New Course form with thumbnail upload
- Lectures Management - per course, list all lectures; Add New Lecture form: title, description, video upload (streams to S3), order
- Books Management - per course, list all books; Add New Book form: title, PDF upload (streams to S3); Delete book
- Students Management - list all registered students, view enrolled courses per student, disable/enable accounts
- Payments - list all payment records: student name, course, amount, payment gateway (Razorpay/Stripe), date, status

Presented By Dream Byte Solutions

- Meetings Management - create new meeting (title, date, time, meeting link or Zoom link), assign to course, delete meeting
- Settings - update site settings: platform name, support email, Razorpay/Stripe key visibility toggle
- Protect all /admin routes - redirect non-admin users to /dashboard

PHASE 7 — Payment Integration (Razorpay + Stripe) — Week 5 | Days 32–36

- Create Payment schema: userId, courseId, gateway (razorpay/stripe), amount, currency, status, transactionId, createdAt
- Razorpay Flow (India): POST /api/payment/razorpay/order — create Razorpay order; frontend opens Razorpay popup; POST /api/payment/razorpay/verify — verify signature with HMAC, create enrollment on success
- Stripe Flow (International): POST /api/payment/stripe/intent — create Stripe PaymentIntent; frontend uses Stripe.js Elements; Stripe Webhook confirms payment; POST /api/payment/stripe/webhook — verify event, create enrollment
- On successful payment of either gateway: create Enrollment record in MongoDB, send confirmation email via SendGrid to student, send notification to admin email
- Build frontend payment UI inside Course Detail Page — two buttons: Pay with UPI/Card (India) and Pay International; show course price in INR and USD
- Handle payment failure gracefully — show error message, allow retry

PHASE 8 — Live Meeting Integration — Week 5–6 | Days 36–40

- Admin creates a meeting from Admin Dashboard: title, description, date, time, meeting URL (Zoom / Google Meet link), assign to course
- Meeting is saved in MongoDB with courseId — only students enrolled in that course can see the meeting
- Student Dashboard shows upcoming meetings in a calendar-style list — title, date, time, Join button
- Join button opens the Zoom / Google Meet link in a new tab (or embedded iframe if allowed)
- Option to integrate Zoom API for automated meeting creation: use Zoom JWT credentials, call Zoom API to create meeting, store meeting ID and join URL automatically
- Send automated email reminders to enrolled students 24 hours and 1 hour before each scheduled meeting

PHASE 9 — Testing, Bug Fixes & Security — Week 6–7 | Days 40–46

- Test all API endpoints in Postman — verify status codes, response shapes, auth protection, and error handling
- Test AWS S3 upload pipeline — verify large video files upload correctly via multipart upload
- Test CloudFront signed URLs — confirm URLs expire correctly and unauthorised users are blocked
- Test full payment flow in sandbox mode — Razorpay test keys + Stripe test mode

Presented By Dream Byte Solutions

- Test student enrollment flow end-to-end — purchase course, get access, watch video, download book
- Test Admin Dashboard — add course, upload video, upload book, create meeting, view students
- Full mobile responsiveness testing — Bootstrap 5 breakpoints on all pages and dashboards
- Security audit — JWT expiry enforcement, rate limiting on auth routes, CORS whitelist for subdomain, Helmet headers, input sanitisation with express-validator
- Cross-browser testing — Chrome, Firefox, Safari, Edge
- Google Lighthouse audit — target 85+ performance on key pages

PHASE 10 — Subdomain Deployment & Go Live (AWS) — Week 7–8 | Days 46–52

- Set up production AWS environment: S3 buckets (videos + books), CloudFront distribution with origin access control, ACM SSL certificate for learn.indianyogaassociation.com
- Configure DNS: Add CNAME record for learn.indianyogaassociation.com pointing to CloudFront distribution or EC2 Elastic IP in the domain registrar's DNS panel
- Deploy Node.js backend to AWS EC2 (Ubuntu): install Node.js, PM2, Nginx; configure Nginx as reverse proxy to Node.js on port 5000; enable SSL via ACM or Certbot
- Build React frontend for production: `npm run build` — upload `dist/` folder to S3 static hosting bucket; invalidate CloudFront cache
- Set all production environment variables on EC2 — MongoDB Atlas URI, JWT secrets, AWS keys, Razorpay/Stripe live keys, SendGrid key
- Run full end-to-end smoke test on live production URL — register, purchase course, watch video, admin login, add course
- Submit sitemap to Google Search Console for learn.indianyogaassociation.com

5. Dashboard Breakdown

5.1 Student / User Dashboard

The student dashboard is the main hub for learners after they log in. URL: learn.indianyogaassociation.com/dashboard

Page / Section	What It Shows	Key Features
My Courses	All enrolled courses with progress bars	Continue learning button, completion percentage
Course Player	Video lecture with sidebar lecture list	Mark complete, prev/next navigation, notes

Presented By Dream Byte Solutions

Books & Materials	Downloadable PDFs per course	Download button (signed S3 URL, enrolled only)
Live Meetings	Upcoming and past meetings per course	Join button, date/time, instructor name
Profile	Account information	Edit name, email, photo, change password
Course Catalog	Browse all available courses	Search, filter by category, enroll / purchase

5.2 Admin Dashboard

The admin dashboard gives full control over all LMS content and users. URL: learn.indianyogaassociation.com/admin

Page / Section	What It Controls	Key Actions
Overview	Platform-wide statistics	Total students, revenue, active courses, new enrollments
Courses	All courses in the system	Add, edit, publish/unpublish, delete course
Lectures	All lectures per course	Upload video (AWS S3), reorder, edit, delete
Books / Materials	All PDFs per course	Upload PDF (AWS S3), rename, delete
Students	All registered users	View enrolled courses, disable/enable account
Payments	All payment records	Filter by date/course/gateway, export CSV
Meetings	Live meeting schedule	Add Zoom/Meet link, assign to course, delete
Settings	Platform configuration	Site name, support email, notification toggles

Presented By Dream Byte Solutions

6. AWS Architecture for Media

All video lectures and course books are stored on AWS infrastructure for security, scalability, and fast delivery. Here is the complete flow:

Video Upload & Storage Flow (Admin → AWS)

- Admin uploads video from Admin Dashboard → Multer receives file on Node.js server → File is streamed via AWS SDK multipart upload to S3 (videos/ prefix)
- S3 triggers an event (S3 Event Notification or manual) → Lambda function or EC2 worker runs FFmpeg to transcode video to HLS format (.m3u8 + .ts segments)
- FFmpeg also generates a thumbnail image (.jpg) from the first frame → All processed files saved back to S3
- Lecture document in MongoDB is updated with: videoUrl (S3 HLS URL), thumbnailUrl, duration

Video Streaming Flow (Student → CloudFront)

- Student clicks 'Watch Lecture' → Frontend calls GET /api/lectures/:id/stream
- Backend checks enrollment: if not enrolled → 403 Forbidden
- If enrolled → AWS SDK generates a signed CloudFront URL valid for 2 hours → URL returned to frontend
- Video.js player loads the signed HLS URL via HLS.js → Video streams from CloudFront edge location nearest to the student

Book / PDF Access Flow

- Admin uploads PDF from Admin Dashboard → Multer streams to S3 (books/ prefix)
- Student clicks 'Download Book' → Backend checks enrollment → Generates signed S3 URL valid for 15 minutes → Student downloads file
- Direct S3 URLs are NEVER exposed to the frontend - all access is via time-limited signed URLs

AWS Service	Purpose	Configuration
S3 Bucket (Videos)	Store raw uploaded videos and HLS processed segments	Private bucket, no public access, CORS enabled for uploads
S3 Bucket (Books)	Store course books and PDF study materials	Private bucket, same account, separate prefix
CloudFront	CDN delivery for video streaming globally	Origin: S3 video bucket, signed URLs enabled
AWS Lambda (optional)	Serverless FFmpeg video processing on upload	Triggered by S3 event on new video upload

Presented By Dream Byte Solutions

EC2 (Backend)	Host Node.js API server	t2.micro / t3.small Ubuntu instance with Nginx + PM2
ACM (SSL)	SSL certificate for subdomain	Free certificate for learn.indianyogaassociation.com
IAM	Access control for S3 and CloudFront	Least-privilege IAM user with only required S3 permissions

7. Complete Folder Structure

File / Folder	Type	Purpose
 lms-client/	React App (Vite)	Full LMS frontend application
 src/		All source code
 components/	Reusable UI	Navbar, Footer, VideoPlayer, CourseCard, Sidebar
 pages/student/	Student Pages	Dashboard, CoursePlayer, Materials, Meetings, Profile
 pages/admin/	Admin Pages	AdminDashboard, CourseManager, LectureManager, BookManager
 pages/auth/	Auth Pages	Login, Register, ForgotPassword, ResetPassword
 pages/public/	Public Pages	CourseCatalog, CourseDetail, Home (LMS landing)
 hooks/	Custom Hooks	useAuth, useCourses, useProgress, usePayment
 context/	State Management	AuthContext, EnrollmentContext
 api/	API Layer	Axios instance + all API call functions
 assets/	Static Files	Images, fonts, SVG icons
 styles/	CSS	Global styles, Bootstrap overrides
 lms-server/	Node.js App	Backend REST API for LMS
 routes/	Express Routes	auth, courses, lectures, books, payments, admin, meetings

Presented By Dream Byte Solutions

models/	Mongoose Schemas	User, Course, Lecture, Book, Enrollment, Payment, Meeting
controllers/	Business Logic	authController, courseController, paymentController, adminController
middleware/	Middleware	authGuard, roleGuard (admin), errorHandler, upload (Multer)
services/	Service Layer	awsService (S3 + signed URLs), emailService, ffmpegService
config/	Configuration	db.js (MongoDB), aws.js (S3 client), cloudfront.js
.env	Environment Vars	DB, JWT, AWS, Razorpay, Stripe, SendGrid secrets
server.js	Entry Point	App initialisation and server start

8. Environment Variables (.env Files)

Server .env File (Backend — Node.js)

```
PORT=5000 NODE_ENV=production
MONGO_URI=mongodb+srv://username:password@cluster.mongodb.net/lms_db
JWT_SECRET=your_super_secret_jwt_key_here JWT_EXPIRE=7d JWT_REFRESH_EXPIRE=30d # AWS
Configuration AWS_ACCESS_KEY_ID=your_aws_access_key
AWS_SECRET_ACCESS_KEY=your_aws_secret_key AWS_REGION=ap-south-1
AWS_S3_VIDEOS_BUCKET=yoga-lms-videos AWS_S3_BOOKS_BUCKET=yoga-lms-books
AWS_CLOUDFRONT_DOMAIN=your-distribution.cloudfront.net
AWS_CLOUDFRONT_KEY_PAIR_ID=your_key_pair_id AWS_CLOUDFRONT_PRIVATE_KEY=-----BEGIN RSA
PRIVATE KEY-----... # Payment Gateways RAZORPAY_KEY_ID=rzp_live_XXXXXXXXXX
RAZORPAY_KEY_SECRET=your_razorpay_secret STRIPE_SECRET_KEY=sk_live_XXXXXXXXXXXXXXXXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXXXXXXXXXXXXXXXX # Email
SENDGRID_API_KEY=SG.XXXXXXXXXXXXXXXXXXXXXXXXXX
EMAIL_FROM=noreply@indianyogaassociation.com
```

Client .env File (Frontend — React)

```
VITE_API_URL=https://api.learn.indianyogaassociation.com/api
VITE_RAZORPAY_KEY_ID=rzp_live_XXXXXXXXXX
VITE_STRIPE_PUBLIC_KEY=pk_live_XXXXXXXXXXXXXXXXXX VITE_CLOUDFRONT_BASE=https://your-
distribution.cloudfront.net
```

Presented By Dream Byte Solutions

9. Key API Endpoints

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register new student account
POST	/api/auth/login	Public	Login, returns JWT
POST	/api/auth/forgot-password	Public	Send reset password email
POST	/api/auth/reset-password	Public	Reset password with token
GET	/api/courses	Public	Get all published courses
GET	/api/courses/:id	Public	Get single course with lectures list
GET	/api/user/enrollments	Student	Get all enrolled courses
POST	/api/user/progress	Student	Update lecture completion
GET	/api/lectures/:id/stream	Student (Enrolled)	Get signed CloudFront URL for video
GET	/api/lectures/:id/book	Student (Enrolled)	Get signed S3 URL for PDF
GET	/api/meetings/course/:id	Student (Enrolled)	Get meetings for a course
POST	/api/payment/razorpay/order	Student	Create Razorpay order
POST	/api/payment/razorpay/verify	Student	Verify Razorpay payment + enroll
POST	/api/payment/stripe/intent	Student	Create Stripe PaymentIntent
POST	/api/payment/stripe/webhook	Stripe	Handle Stripe payment events
GET	/api/admin/stats	Admin	Platform overview statistics
POST	/api/admin/courses	Admin	Create new course
PUT	/api/admin/courses/:id	Admin	Edit course details
DELETE	/api/admin/courses/:id	Admin	Delete course
POST	/api/admin/lectures	Admin	Add lecture + upload video to S3
PUT	/api/admin/lectures/:id	Admin	Edit lecture
DELETE	/api/admin/lectures/:id	Admin	Delete lecture + remove from S3
POST	/api/admin/books	Admin	Upload course book PDF to S3
DELETE	/api/admin/books/:id	Admin	Delete book + remove from S3
POST	/api/admin/meetings	Admin	Create new meeting
DELETE	/api/admin/meetings/:id	Admin	Delete meeting

Presented By Dream Byte Solutions

GET	/api/admin/students	Admin	List all students
GET	/api/admin/payments	Admin	List all payment records

10. Estimated Cost & Required Services

Layer / Category	Technology / Plan	Free Tier Limit	Paid Plan (After Free Tier)
Domain / Subdomain	learn.indianyogaassociation.com	Free - subdomain of existing domain	No extra cost - just a DNS record
MongoDB Atlas	M0 Free Cluster → M2 Paid	512 MB storage	M2 - \$9/mo M5 - \$25/mo
AWS S3 (Videos)	AWS S3 Standard	5 GB + 20,000 GET requests/month free	~\$0.023/GB/month + \$0.09/GB transfer
AWS CloudFront	CloudFront CDN	1 TB transfer/month free (12 months)	~\$0.085/GB after free tier
AWS EC2 (Backend)	t2.micro / t3.small	t2.micro free 12 months	t3.small - ~\$15–\$20/month
AWS Lambda (FFmpeg)	Lambda + FFmpeg Layer	1M requests/month free forever	~\$0.20 per 1M requests
SSL Certificate	AWS ACM	Free forever	Free - included with CloudFront
Razorpay (India Pay)	No monthly fee	No limit - pay per transaction	2% per transaction (always)
Stripe (Intl. Pay)	No monthly fee	No limit - pay per transaction	2.9% + \$0.30 per transaction
SendGrid (Email)	Free Tier	100 emails/day forever	Essentials - \$19.95/month (50K emails)
Zoom API (Meetings)	Basic Plan	40-min limit on free meetings	Pro - \$13.33/host/month (unlimited)
TOTAL STARTUP COST	~\$0 / ₹0 to launch	All free tiers sufficient for launch	~\$30–\$60/month as platform grows

Presented By Dream Byte Solutions

11. Project Summary

This project delivers a complete, production-grade Learning Management System (LMS) on the subdomain learn.indianyogaassociation.com. It is built for scalability, security, and a great student learning experience.

Key Project Facts at a Glance:

- Full Tech Stack: React 18 + Vite, Bootstrap 5, Framer Motion, Node.js, Express, MongoDB, Mongoose, JWT, AWS S3, AWS CloudFront, FFmpeg, Razorpay, Stripe, SendGrid, Zoom API
- Total Development Phases: 10 phases from Planning through to Live Deployment on AWS
- Estimated Timeline: 6–8 weeks solo developer | 4–5 weeks with a team of 2–3 developers
- Two Dashboards: Full-featured Student Dashboard and complete Admin Dashboard with content management
- Secure Media: All videos and books stored on AWS S3 - only accessible via time-limited signed URLs for enrolled students
- Payment Gateways: Razorpay for Indian students (INR) + Stripe for international students (USD)
- Total API Endpoints: 28+ REST API routes covering auth, courses, lectures, books, payments, meetings, and admin
- Subdomain Deployment: Hosted entirely on AWS (EC2 + S3 + CloudFront) on learn.indianyogaassociation.com
- Startup Cost: Near zero - all AWS free tiers and third-party free plans are sufficient to launch

Presented By Dream Byte Solutions

Revision Policy — Please Read Carefully

After the test deployment is delivered, you are entitled to 2 free revision rounds.

- ✓ 1st Revision - Completely FREE. Share all feedback clearly.
- ✓ 2nd Revision - Completely FREE. Last complimentary round.
- ✗ 3rd Revision onwards - Chargeable. Will be billed as additional work.

To make the most of your free revisions — review all pages thoroughly, consolidate all feedback into one list, and share it clearly in each round. 3rd revision onwards = chargeable.

— Document Complete. Happy Coding!  —

